ALC: Adaptive Lossless Compression for LLM Checkpoints

Wenqian Li†, Li Peng†‡, Weiran Liu†, Liqiang Peng†, Zhiqiang Lyu†, Yan Zhang†,
Wei Wu†, Lei Zhang†, Jiamang Wang†, Lin Qu†, and Jinfei Liu‡
†Alibaba Group, ‡Zhejiang University
†liwq62@gmail.com, {jerry.pl, weiran.lwr, plq270998, zhiqiang.lv, zy143424, wei.wu}@alibaba-inc.com,
zongchao.zl@taobao.com, jiamang.wang@alibaba-inc.com, xide.ql@taobao.com
‡jinfeiliu@zju.edu.cn

***Abstract*—The storage footprint of large language model (LLM) training checkpoints is growing rapidly, posing a major challenge to production storage infrastructure. For checkpoint archival, strict bit-exactness is indispensable to guarantee storage reliability and exact training resumption. However, existing compression methods remain inadequate: lossy approaches compromise data integrity and faithful recovery, while existing lossless methods fail to fully exploit the structural and temporal redundancies inherent in training checkpoints.**

In this paper, we present a systematic characterization of LLM checkpoints and identify three key properties that determine their compressibility: the deterministic reconstructability of mixed-precision weights, the highly heterogeneous temporal correlations across parameter types, and the severe sign-flip penalty that undermines conventional delta encoding. Motivated by these observations, we propose ALC, an adaptive lossless compression framework for LLM checkpoints. At its core, ALC introduces RotDelta, a novel sign-agnostic delta encoding algorithm that mitigates the sign-flip penalty. To further address structural and temporal heterogeneity, ALC employs an adaptive, cost-model-driven strategy that selects the most effective encoding path for each tensor. Combining redundancy removal for derivable states with compression, ALC maximizes checkpoint storage savings. Extensive evaluations across diverse LLM architectures demonstrate that our approach consistently outperforms state-of-the-art (SOTA) lossless baselines, reducing raw checkpoint footprints by 41.9% for Qwen3-MoE-235B while preserving bit-exactness.

I. Introduction

Large language models (LLMs) have demonstrated remarkable capabilities through training pipelines that span large-scale pre-training, domain-specific fine-tuning, and reinforcement learning. Due to the frequently occured training failures and long duration of these training runs, developers periodically save complete snapshots of the training state, known as checkpoints [1]–[6]. These snapshots are essential for resuming training after failures [1], [7], evaluating intermediate models, and ensuring historical reproducibility, as shown in Figure 1.

While essential for training reliability, this routine checkpointing inevitably translates into substantial storage footprints. To ensure reproducibility and fault tolerance across distributed clusters in production environments, massive checkpoints are periodically offloaded to persistent remote storage, such as Hadoop Distributed File System (HDFS), or Network-Attached Storage (NAS) based on task characteristics and personal preferences [1], [6], [8]. Since modern LLM training predominantly relies on Adam-family optimizers [9], [10], a complete checkpoint must archive extensive optimizer states (*e.g.*, first- and second-order moments) in addition to model parameters, expanding the on-disk footprint to roughly 7× the size of the active BF16 weights. Furthermore, training pipelines retain many historical checkpoints across training steps for model selection and retrospective evaluation. As a result, aggregate storage demands rapidly escalate to the petabyte scale as model size and training tasks expands. As illustrated in Figure 2, the average quarterly checkpoint volume on Alibaba's internal storage platform has grown by over 4× in less than two years, from approximately 89 PB in 2024 Q3 to over 366 PB in 2026 Q1. Amplified by surging storage prices, this explosive growth places an unsustainable financial burden. Consequently, checkpoint storage compression has emerged as a critical infrastructure requirement.

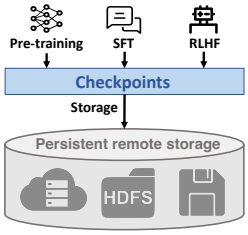


Fig. 1: The generation and storage of checkpoints.

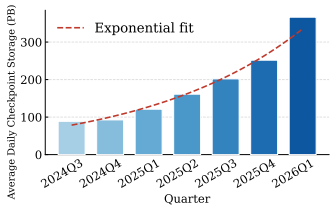


Fig. 2: Quarterly average daily checkpoint volume on Alibaba's internal platform (2024 Q3–2026 Q1).

Existing compression techniques generally partition into lossy and lossless paradigms, yet none completely satisfy the dual requirements of strict data fidelity and high compression efficiency demanded by LLM checkpoint archival. As the foundational data layer, storage infrastructure is obligated to provide users with completely intact, uncorrupted data [11]–[13]. Lossy checkpoint compressors, such as ExCP[14], Inshrinkerator[15], and Amber [16], intentionally introduce numerical errors to reduce file sizes. This approach directly violates the fundamental lossless guarantee of storage systems, introducing potentially negative impacts on the training dynamics and deterministic reproducibility during training resumption without user awareness [17]–[19]. Conversely,

existing model-aware compressors (*e.g.*, ZipNN[20], FM-Delta[21] and ZipLLM [22]) and general-purpose compressors (*e.g.*, Zstd[23] and Gzip[24]) fail to capture the unique structural and temporal characteristics of training checkpoints (see §III), and therefore do not fully exploit their compression potential. As a result, existing methods either sacrifice the data-integrity guarantee required by storage systems or leave substantial compression opportunities untapped across the full checkpoint footprint.

Contributions. To bridge this gap, we systematically analyze LLM checkpoints and identify unique structural properties of checkpoints that shape our compression design. Building on these observations, we propose a novel sign-agnostic delta compression algorithm RotDelta and an adaptive compression framework ALC that achieves improved compression efficiency while maintaining strict losslessness. Our contributions are summarized as follows:

1) **Checkpoint characterization (§III).** To motivate our design, we present a comprehensive characterization of LLM training checkpoints, revealing the unique structural properties and compression potential of checkpoint tensors. We identify three key properties that govern checkpoint compressibility: the mixed-precision bit-exact redundancy, the temporal correlation divergence, and the compression bottleneck induced by sign flips penalty.
2) **Sign-agnostic delta compression algorithm (§IV).** To resolve the sign-flip penalty above, we design RotDelta, a sign-agnostic delta compression algorithm that achieves more efficient compression under sign-flip. By relocating the sign bit before differencing, RotDelta restricts the residual magnitude to the difference rather than the sum, leading to improved compression efficiency.
3) **Adaptive compression selection framework (§V).** To exploit the temporal heterogeneities of checkpoints, we design a cost-driven adaptive selection framework ALC that resolves the inherent heterogenous tensor distributions in LLM checkpoints. The framework dynamically determines the optimal compression path (direct encoding, PCDelta, or our proposed RotDelta) for each tensor based on fast entropy estimation. Complemented by a redundancy-removal pass for derivable states, this framework ensures maximized checkpoint storage savings.
4) **End-to-end evaluation (§VI).** We evaluate ALC using checkpoints derived from training multiple LLMs across diverse architectures and scales. Compared to SOTA lossless compressors, our approach achieve the overall compression ratio of approximately 40%. Meanwhile, ALC remains strictly lossless, preserves training reproducibility, and achieves a compression throughput of 198 MB/s alongside a decompression throughput of 232 MB/s. We show that compressing checkpoint in backup storage system is economically beneficial.

II. Related Work

This section reviews existing techniques for reducing storage overhead, categorized into checkpoint, model, and general-purpose compression. While lossy checkpointing may disrupt training stability, lossless model and general-purpose compressors do not account for training-specific dynamics like temporal decay and frequent optimizer sign flips. These limitations motivate us to present a bit-exact, training-independent lossless checkpointing scheme that adaptively optimizes these heterogeneous states to achieve more efficient checkpoint storage.

A. Checkpoint Compression

To mitigate checkpoint storage overhead, existing approaches aim at lossy compression. ExCP [14] combines non-uniform quantization with joint weight-momentum pruning on checkpoint residuals. Inshrinkerator [15] operates as an in-training compression system that dynamically searches for optimal non-uniform quantization configurations and rearranges parameters to enhance delta encoding. Meanwhile, Amber [16] performs approximate incremental checkpointing by preserving only the parameters flagged as changed between successive checkpoints. Earlier works on DNN and recommendation training, including LC-Checkpoint [25], Check-N-Run [26], Delta-DNN [27], and SSDC [28], explore exponent-based quantization, incremental checkpointing, error-bounded lossy deltas, and sparse differentials at lower model scales. However, these methods are inherently lossy and require tight coupling with the active training runtime. Such loss of data fidelity and training-side dependency render them unsuitable for robust, independent checkpoint storage services that that must guarantee bit-level integrity of stored data.

B. Model Compression

Multiple lossless compression methods for model storage reduction have been proposed. ZipNN [20] separates IEEE 754 floating point byte fields and Huffman-codes the exponent stream of model parameters. FM-Delta [21] employs a delta encoding ideas to exploit the similarity of fine-tuned model on the storage reduction. It maps each float to an integer through a piecewise-continuous monotonic map and entropy-codes the integer-domain delta between base and fine-tuned weights. ZipLLM [22] entropy-codes the bitwise XOR residuals between fine-tuned and base weights with Zstd and achieves a competitive model storage reduction over FM-Delta [21]. None of these model compressors considered the specific cross-checkpoint characater of optimier states tensors in checkpoints (as identified §III), and therefore limits its effectiveness on checkpoints storage reduction.

C. General-purpose Compression

General-purpose coders (Huffman [29], arithmetic/range coding [30], [31], Gzip [24], Zstd [23]) and format-aware floating-point compressors (FPZIP [32], FPC [33], Bitshuffle [34]) treat tensors as opaque streams or exploit only the IEEE 754 layout, yielding limited gains on LLM-related model data, as noted in [21]. Delta compression for general backup

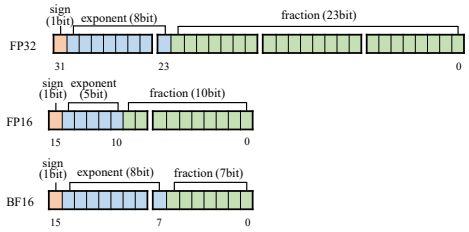


Fig. 3: Bit field layout of the three floating point formats used in mixed precision training.

storage has been explored in the context of file systems and databases [35], [36], but these methods failed in expolit the numerical characteristics of floating-point tensors and limits their applicability to checkpoints.

III. Analysis of LLM Training Checkpoints

We characterize LLM training checkpoints to guide lossless compression, covering checkpoint composition (§III-B), the data characteristics of a single checkpoint (§III-C), and the correlation across a sequence of checkpoints (§III-D). These investigations reveal that no single scheme can optimally handle all tensors, motivating the adaptive selection framework ALC and our sign-agnostic RotDelta algorithm.

A. Preliminaries

1) Floating Point Representation: In typical training workflows, internal states are formulated as floating-point tensors. To balance throughput and numerical stability, modern large-scale training commonly adopts mixed-precision strategies [37], [38]: numerically sensitive master weights and optimizer momenta are maintained in FP32, while forward/backward computations and model weights use reduced formats like BF16 or FP16 to achieve higher throughput and lower memory cost.

Under the IEEE 754 standard, each floating-point number is composed of three bit fields: a sign bit s, an exponent e, and a mantissa m, which together encode the value $(-1)^s \times 2^{e-\text{bias}} \times (1.m)$. FP32 allocates 1, 8, and 23 bits to these fields respectively. BF16 retains the 8-bit exponent for overflow robustness while shortening the mantissa to 7 bits, and FP16 narrows the exponent to 5 bits in exchange for a 10-bit mantissa, as illustrated in Figure 3.

2) Training Checkpoints: Training checkpoints are a periodically saved snapshot of the training state that serves three main purposes. First, for fault tolerance, it allows long-running training to resume from the latest state after hardware or software failures, avoiding restarts from scratch [3], [26], [39]. Second, for model selection, intermediate checkpoints let developers evaluate and select the best model before training ends. Third, for traceability, they record the optimization path to ensure training reproducibility [1], [40].

B. Checkpoint Composition

Considering the Adam-family optimizers [9], [10] are predominantly used in LLM training, we take the Adam optimizer as an example in our following analysis. Under mixed-precision training [37], a checkpoint for an M-parameter model comprises $12M$ bytes of FP32 states (master weights and optimizer moments m_t, v_t at $4M$ each) and $2M$ bytes of BF16 compute weights. Figure 4 illustrates this composition across the Qwen3.5 family: the FP32 portion consistently dominates, accounting for $\sim$85% of the on-disk footprint (e.g., 352 GB for Qwen3.5-27B), while the BF16 copy occupies the remaining $\sim$15%.

Mixed-precision structural redundancy. Beyond the size-level partition above, the FP32 master weights and the BF16 compute weights are not independent: by the IEEE 754 standard [41], the BF16 weight is obtained by truncating its FP32 master to the top 16 bits,

$$W_{\text{bf16}} = \text{truncate}_{16}(W_{\text{fp32}}), \tag{1}$$

Because this mapping is bit-exact and platform-independent, the BF16 copy can be fully reconstructed from its FP32 master. ***Findings.*** Owing to the structural redundancy inherent in mixed-precision training, the BF16 compute weights is bit-exactly determined by its FP32 master weights via (1) and can be removed directly from storage without compromising losslessness. This leads to a 15% reduction in checkpoint size.

C. Checkpoint Characteristics

The three parameter types (weights, m_t, v_t) follow different update rules and therefore exhibit distinct value distributions within a single checkpoint. As shown in Figure 6, all three are sharply concentrated near zero. They differ in sign and dynamic range: weights and m_t are approximately symmetric around zero, while v_t is strictly non-negative and concentrated at small positive values. In particular, m_t occupies a much narrower magnitude range than the weights, so its updates more frequently cross zero and produce sign flips, a phenomenon we revisit in §III-D2.

At the byte level, Figure 5 shows that entropy varies widely across the four bytes of an FP32 element. For the sign bit, v_t generates 0 bit of entropy because it is always positive, while weights and m_t yield 1 bit due to mixed signs. For the remaining bytes, the exponent carries much less entropy than the mantissa bytes, which approach 8 bits and are nearly uniform. This contrast holds across all three parameter types. ***Findings.*** Across both views, the three FP32 parameters share concentration near zero but diverge in sign symmetry and dynamic range. This tensor-level heterogeneity makes any single fixed encoder suboptimal and motivates the adaptive selection scheme of §V-D. At the byte level, the exponent byte holds substantial compression potential while the mantissa

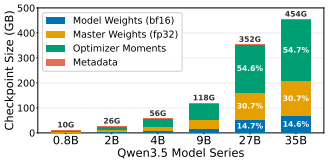


Fig. 4: Checkpoint storage breakdown across the Qwen3.5 family.

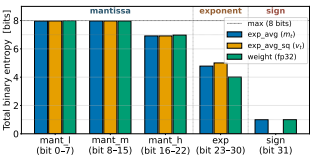


Fig. 5: Per-byte Shannon entropy of m_t, v_t and weights in a checkpoint.

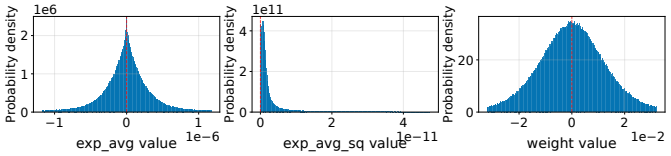


Fig. 6: Value distributions of m_t, v_t and model weights.

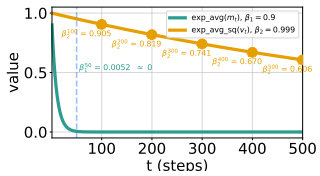


Fig. 7: Decay rates of Adam's m_t and v_t.

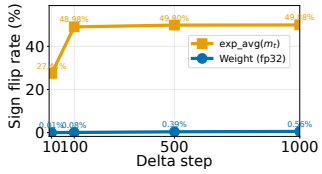


Fig. 8: Sign-flip rates of m_t and weights across training steps.

bytes are close to incompressible, and the sign-bit entropy varies with parameter type, which directly shapes the design of our encoding scheme in §IV-B.

D. Cross-Checkpoint Correlation

Beyond the per-checkpoint statistics of §III-C, the temporal evolution of checkpoints governs how effective delta encoding can be. Let $\mathbf{x}_t$ and $\mathbf{x}_{t-n}$ denote the tensor states at training steps t and $t-n$, where n is the checkpoint storage interval. The efficacy of delta encoding is determined by the entropy of the delta $\mathbf{\Delta}_{t-n} = \mathbf{x}_t - \mathbf{x}_{t-n}$: smaller deltas yield lower entropy and thus tighter compression. We therefore characterize cross-checkpoint behavior along three axes: how the correlation between $\mathbf{x}_t$ and $\mathbf{x}_{t-n}$ decays with the step interval n (§III-D1), how frequently signs flip across step intervals (§III-D2), and why such flips form a fundamental bottleneck for existing delta encoders (§III-D3).

1) Temporal Correlation Decay: We study the temporal evolution of checkpoints over training steps. Operating with standard hyperparameters ($\beta_1 = 0.9, \beta_2 = 0.999$), Adam maintains the first-moment estimate m_t and second-moment estimate v_t as exponential moving averages of the gradients g_t and their squares g_t^2:

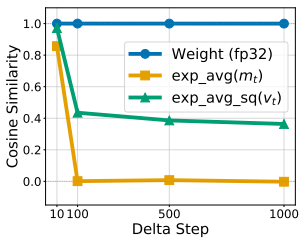
$$m_t = \beta_1^n m_{t-n} + (1-\beta_1)\sum_{i=0}^{n-1} \beta_1^i g_{t-i}, \tag{2}$$

$$v_t = \beta_2^n v_{t-n} + (1-\beta_2)\sum_{i=0}^{n-1} \beta_2^i g_{t-i}^2. \tag{3}$$

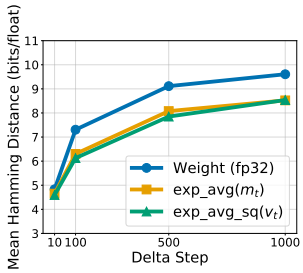
The coefficient β^n quantifies the correlation between the current optimizer state and its value n steps earlier. As shown in Figure 7, the two moments decay at very different rates: m_t loses correlation within tens of steps ($\beta_1^{50} \approx 0.005$), while v_t remains correlated over hundreds of steps ($\beta_2^{500} \approx 0.606$).

To empirically validate the above analysis, we measure two complementary metrics *cosine similarity* and *bit-level distance* across step intervals $n \in \{10, 100, 500, 1000\}$.

As Figure 9 confirms, across all parameters, cosine similarity decreases and bit-level distance increases monotonically with n. The two Adam moments track their theoretical β^n decay



(a) Cosine similarity



(b) Bit-level distance

Fig. 9: Cosine similarity and bit-level distance vs. step interval n, measured for m_t, v_t, and model weights.

tightly: m_t decorrelates rapidly (dropping near zero by $n \approx 100$, where $\beta_1^{100} \approx 2.7 \times 10^{-5}$), while v_t decays gradually, falling to 0.364 at n=1000 (cf. $\beta_2^{1000} \approx 0.368$). Model weights behave differently: their cosine similarity remains extremely high (≥ 0.9998), yet they exhibit the largest bit-level distance. This occurs because weights concentrate near zero, and tiny relative updates alter low-order mantissa bits of IEEE 754 representations without rotating the weight vector. These distinct profiles motivate per-tensor compression strategies.

2) Sign-Flip Characteristics: Beyond magnitude decay, we also examine sign flips (Figure 8), because subtracting two opposite-signed values yields a delta of magnitude $|x_t| + |x_{t-n}|$ rather than $\big||x_t| - |x_{t-n}|\big|$. This breaks the small-delta assumption of delta encoding and inflates the entropy of the delta stream. The sign-flip rate is defined as $\eta = \Pr[\mathrm{sgn}(x_t) \neq \mathrm{sgn}(x_{t-n})]$. Since v_t is always positive, its sign never changes regardless of n. Model weights are also stable, with sign-flip rates below 1% throughout training. In contrast, m_t flips much more often: as β_1^n decays, the rate rises to about 50% for large n, close to a random flip. Yet no existing delta encoder explicitly handles sign flips or mitigates the compression penalty they cause.

3) Compression Bottleneck under Sign Flips: §III-D2 showed that m_t exhibits a sign-flip rate approaching 50% at large step intervals. Here we perform a quantitative analysis to reveal why sign flips constitute a fundamental compression bottleneck for delta encoding.

Sign-magnitude decomposition. Each value can be written as $x = s \cdot k$, where $s \in \{+1, -1\}$ is the sign and $k = |x|$ is the magnitude. The delta between the two values is

$$D_t = x_t - x_{t-n} = s_t k_t - s_{t-n} k_{t-n}. \tag{4}$$

Magnitude change. The difference magnitude $|D_t|$ depends on whether a sign flip occurs:

$$|D_t| = \begin{cases} |k_t - k_{t-n}|, & s_t = s_{t-n}, \\ k_t + k_{t-n}, & s_t \neq s_{t-n}. \end{cases} \tag{5}$$

Without a flip, $|D_t|$ is a small difference and has low entropy. With a flip, $|D_t|$ jumps to the magnitude sum. Then, we quantify

this gap by the *encoding penalty*

$$\mathcal{K} = \frac{\mathbb{E}[|D_t| \mid I_{\text{flip}}{=}1]}{\mathbb{E}[|D_t| \mid I_{\text{flip}}{=}0]}. \tag{6}$$

where $I_{\text{flip}}{=}1$ indicates a sign flip and $I_{\text{flip}}{=}0$ indicates no flip. The encoding penalty $\mathcal{K}$ measures how many times larger the delta is when a sign flip occurs compared to when it does not. Since magnitudes change little between consecutive checkpoints (§III-D1) yet sign flips occur frequently for m_t (§III-D2), $k_t + k_{t-n} \gg |k_t - k_{t-n}|$ and thus $\mathcal{K} \gg 1$, meaning that sign flips dominate the m_t delta entropy.

Findings. The three parameters (m_t, v_t, and weights) show markedly different cross-checkpoint decay patterns: m_t decorrelates within tens of steps, v_t remains correlated over hundreds of steps, and weights stay close in direction throughout training. This heterogeneity favors direct compression for weakly correlated parameters and delta compression for strongly correlated ones. Their magnitude and sign behavior also differ: m_t has small magnitudes but a sign-flip rate of about 50% at large n, v_t never flips, and weights change little in magnitude and flip below 1%. Because sign flips inflate the delta from $|k_t - k_{t-n}|$ to $k_t + k_{t-n}|$, decoupling sign from magnitude is the key design goal of §IV.

IV. RotDelta: An Improved Sign-Agnostic Delta-Compression Algorithm

Building on the analysis of §III, this section proposes RotDelta, a sign-agnostic delta-compression algorithm that eliminates the sign-flip penalty. §IV-A identified sign-magnitude decoupling as the key to removing the sign-flip penalty in delta encoding. §IV-B then presents the RotDelta algorithm, which realizes sign-magnitude decoupling through a preprocessing map called Δ-RotMap. §IV-C derives a unified upper bound on its delta bit-width that holds regardless of sign flips, unlike existing schemes whose bit-cost grows much larger under sign flips.

A. Limits of Existing Delta Encoding

No prior delta encoder is specifically designed for the temporal sequence of training checkpoints. Existing schemes [21], [22] were designed for fine-tuned model storage and compute integer-domain deltas between two model versions, entropy-coding the result. We analyze FM-Delta [21] as the representative case: it reinterprets each float as an integer via a piecewise-continuous monotonic mapping (*PCMap*) that preserves natural ordering, then entropy-codes the resulting integer delta, hereafter denoted Δ_{PCMap}. The MSB-set behavior we expose below also arises in any XOR-style delta such as ZipLLM [22].

Limitations under Sign Flips. FM-Delta exemplifies the sign-magnitude coupling concretely. Let r_{PCMap} denote the effective bit-width of FM-Delta's integer delta Δ_{PCMap}. Its upper bound on bit width:

$$\begin{aligned} r_{\text{PCMap}} &= \left\lceil \log_2 \left(2^{(n_e+n_m)}(s_p \oplus s_c) + 2^{n_m}(e_p - e_c) + \hat{m}_p - \hat{m}_c \right) \right\rceil \\ &\le \begin{cases} \left\lceil n_m + \log_2 \left(\log_2 \left(\frac{w_p}{w_c} \right) + 2 \right) \right\rceil, & s_p \oplus s_c = 0 \\ n_s + n_e + n_m, & s_p \oplus s_c = 1 \end{cases} \end{aligned} \tag{7}$$

where n_s, n_e, n_m are the sign, exponent, and mantissa bit widths, subscripts p and c denote the previous and current values, $\hat{m}_p$ and $\hat{m}_c$ are the mantissa integers, and $\oplus$ denotes bitwise XOR. When $s_p \oplus s_c = 1$, the bound saturates at $n_s + n_e + n_m$, i.e., the delta can span up to the full IEEE 754 layout regardless of the underlying numerical change. The same MSB-set behavior arises in any XOR-style delta under a sign flip. This degrades the compression gain of delta encoding.

Root cause and resolution. The bit-level bound above stems from *sign-magnitude coupling*: existing schemes difference the floating-point word as a single integer, so opposite-sign pairs scale with the magnitude sum rather than the magnitude difference. Decoupling sign from magnitude bounds the total entropy by

$$H(s_t, |k_t - k_{t-n}|) \le H(s_t) + H(|k_t - k_{t-n}|), \tag{8}$$

where $H(s_t) \le 1$ bit and $H(|k_t - k_{t-n}|)$ stays small even during flips. We realize this decoupling in §IV-B.

B. RotDelta: Sign-Agnostic Delta Compression Algorithm

To address the frequent sign flips that drive the bound above to saturate, we propose **RotDelta**. The core idea of RotDelta is to relocate the sign bit before computing the delta, moving it from the most significant bit (MSB) to the least significant bit (LSB) to mitigate its magnitude-disrupting effect. This rotation, termed *RotMap*, aligns positive and negative values on a single contiguous axis. Consequently, the delta between opposite-sign pairs is then governed by the magnitude difference rather than the magnitude sum, so it remains small whenever the underlying values are close, even if the sign flips. To implement this losslessly, floating-point numbers are reinterpreted to their identical integer representations at the bit level before differencing. Finally, the resulting deltas often contain many leading zeros because the magnitude differences are typically small, so we employ a Leading-Zero Encoder to exploit this sparsity. The full procedure is summarized in Algorithm 1, and we elaborate each stage below.

Algorithm 1: RotDelta: Sign-Agnostic Delta Compression

Input: two checkpoints $\mathcal{C}_p$, $\mathcal{C}_c$; tANS encoder $\mathcal{M}$
Output: compressed $\hat{\mathcal{C}}_p$; uncompressed $\mathcal{C}_c$
1: **for each** tensor pair $(\mathbf{x}_p, \mathbf{x}_c)$ from $(\mathcal{C}_p, \mathcal{C}_c)$ **do**
2: **for each** element pair (x_p, x_c) from $(\mathbf{x}_p, \mathbf{x}_c)$ **do**
3: $u_p \leftarrow \mathrm{RotMap}(x_p)$; $u_c \leftarrow \mathrm{RotMap}(x_c)$
4: $S \leftarrow \mathrm{sgn}(u_c - u_p)$; $\Delta_{\text{RotMap}} \leftarrow |u_c - u_p|$
5: $K \leftarrow \lfloor \log_2 \Delta_{\text{RotMap}} \rfloor$
6: $\mathrm{cp} \leftarrow \mathrm{cp} \,\|\, \mathcal{M}.\mathrm{encode}(S, K)$ ▷ *tANS-encode the joint symbol*
7: $\mathrm{rr} \leftarrow \mathrm{rr} \,\|\, (\Delta_{\text{RotMap}} \bmod 2^K)$ ▷ *lowest K bits, raw*
8: **end for**
9: $\hat{\mathbf{x}}_p \leftarrow \mathrm{cp} \,\|\, \mathrm{rr}$ ▷ cp*: compressed part;* rr*: raw remainder*
10: **end for**
11: $\hat{\mathcal{C}}_p \leftarrow \{\hat{\mathbf{x}}_p\}_{\mathbf{x}_p \in \mathcal{C}_p}$
12: **return** compressed $\hat{\mathcal{C}}_p$; uncompressed $\mathcal{C}_c$

Preprocessing Map. The preprocessing stage of RotDelta (Algorithm 1, Lines 3–4), abbreviated Δ-RotMap, realizes the sign-magnitude decoupling. Before encoding, we apply the Δ-RotMap transformation to shrink the delta and improve compressibility. It converts each pair of corresponding tensors $(\mathbf{x}_p, \mathbf{x}_c)$ from two consecutive checkpoints into a stream of lossless deltas, leaving more compression room for the downstream entropy encoder. For each element pair (x_p, x_c), we reinterpret the IEEE 754 bit pattern as an unsigned integer $U(x)$ and apply a single-bit left rotation:

$$\mathrm{RotMap}(x) = \big((U(x) \ll 1) \mid (U(x) \gg (b{-}1))\big) \bmod 2^b, \tag{9}$$

where b=32 for FP32 and b=16 for BF16. This moves the sign bit from bit $b{-}1$ to bit 0, yielding $u_p = \mathrm{RotMap}(x_p)$ and $u_c = \mathrm{RotMap}(x_c)$. We then compute the absolute difference:

$$\Delta_{\text{RotMap}} = |u_c - u_p|. \tag{10}$$

Because RotMap relocates the sign bit to the LSB, opposite-sign pairs no longer fall on opposite sides of 2^{b-1}, so the delta is governed by the magnitude difference rather than the magnitude sum and remains small whenever the underlying values are close. This shrinkage of Δ_{RotMap} enlarges the compression room for the downstream encoder.

Leading-Zero Encoder. The Leading-Zero Encoder is the encoding stage of RotDelta (Algorithm 1, Lines 5–7). It takes the stream of Δ_{RotMap} values produced by the preprocessing stage and emits a compressed bit-string. Note that $\Delta_{\text{RotMap}} \geq 0$ by (10). Once RotMap removes the sign-flip penalty, the resulting deltas are concentrated near zero, exposing many leading zeros in binary representations. We thus encode the position of the leading non-zero bit and concatenate the remaining low-order bits as a raw bit-stream, inspired by [21].

For each delta Δ_{RotMap}, we extract two parts: a joint symbol (S, K) and a raw remainder. Here $S = \mathrm{sgn}(u_c - u_p)$ is the sign of the integer delta and $K = \lfloor \log_2 \Delta_{\text{RotMap}} \rfloor$ is the zero-indexed position of the leading non-zero bit. The leading non-zero bit at position K is implicitly one and requires no transmission; the lowest K bits of Δ_{RotMap} are stored as raw bits. Notably, the joint symbol (S, K) has a small alphabet of $2b{+}1$ states where b is the bit-width (e.g., $2{\times}32{+}1 = 65$ for FP32) and captures the statistical coupling between update direction and magnitude. We compress it close to the joint entropy $H(S, K)$ using tANS [42] compressor.

Why tANS? We choose tANS over traditional arithmetic/entropy coding (*e.g.*, range coding used in [21]) because it achieves a compression ratio extremely close to the Shannon entropy limit while delivering significantly faster execution speeds, a performance advantage that is particularly pronounced when operating on relatively small alphabets like our symbol space (typically lower than 65).

C. Sign-Agnostic Bit-Width Bound

To quantify RotDelta's gain under sign flips, we derive a bit-width upper bound for Δ_{RotMap} by separately analyzing the opposite-sign and same-sign cases. We use FP32 as the running example for the derivation. Let r_{RotMap} denote the effective bit-width of Δ_{RotMap}, defined similarly to r_{PCMap} in Equation 7. Let $L(x) = U(x) \bmod 2^{31}$ denote the 31-bit magnitude field, the IEEE 754 biased exponent $E(x)$ concatenated with the mantissa $M(x)$, so that $u = \mathrm{RotMap}(x) = 2L(x) + s$.

1) Opposite-sign scenario: Assume WLOG $u_p \geq u_c$. For RotDelta, $\Delta_{\text{RotMap}} = u_p - u_c = 2(L(x_p) - L(x_c)) + (s_p - s_c)$, scaling with the *magnitude difference*. FM-Delta yields $\Delta_{\text{PCMap}} = L(x_p) + L(x_c) + 1$, scaling with the *magnitude sum*. Figure 10 illustrates this on $(-0.5, +2.7)$: Δ-PCMap maps the points to opposite sides of 2^{31}, whereas Δ-RotMap maps them adjacent.

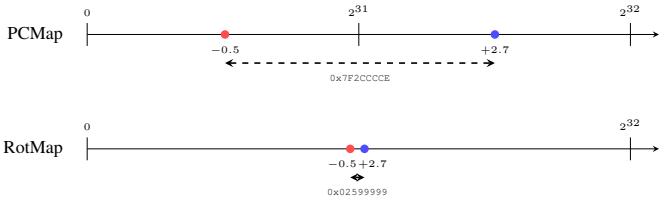


Fig. 10: Integer mapping of the sign-flip pair $(-0.5, +2.7)$ under Δ-PCMap and Δ-RotMap.

Proposition 1. *For any opposite-sign pair (x_p, x_c) whose magnitude ratio falls within the range typical of deep-learning training, Δ_{RotMap} is strictly smaller than Δ_{PCMap}.*

This proposition formally establishes the superiority of RotDelta over conventional integer-domain delta encoders [21] under sign-flip conditions. The proof is in Appendix A in the full version of our paper [43].

2) Same-sign scenario: The shared LSB sign bit cancels under subtraction, $\Delta_{\text{RotMap}} = 2|U(x_p) - U(x_c)|$, a 1-bit left shift of the same-sign Δ_{PCMap}. The effective bit-width is therefore exactly one greater than that of Δ_{PCMap}.

3) Unified bound: Combining both cases, the worst case is dominated by the same-sign branch: opposite-sign deltas are strictly smaller (Proposition 1), while same-sign deltas incur a constant 1-bit overhead. Adding this overhead to the same-sign branch of Equation 7 yields a unified, sign-agnostic upper bound

$$r_{\text{RotMap}} \leq \left\lceil n_m + 1 + \log_2\left(\log_2\left(\frac{\max(|x_p|, |x_c|)}{\min(|x_p|, |x_c|)}\right) + 2\right)\right\rceil \tag{11}$$

which depends solely on the magnitude ratio. Equivalently, Δ-RotMap eliminates Δ-PCMap's $(n_s + n_e + n_m)$-bit sign-flip case and replaces the two-branch bound with a single unified bound.

V. ALC: Adaptive Compression Selection Framework

As established in §III, no single compression scheme is universally optimal across the heterogeneous tensors of an LLM checkpoints. To address this challenge, we propose ALC, an adaptive compression framework that chooses the lowest-cost path for each tensor from three candidates based on deterministic subsample-based cost estimation. ALC integrates one direct path (I-Align) and two delta paths (PCMap and RotMap) under a shared tANS-based encoding framework. §V-A presents the overall three-phase pipeline, §V-B describes the elimination of losslessly recoverable redundancy, §V-C and §V-D detail the candidate paths and the cost-driven selector.

A. Overview

We propose a checkpoint compression framework ALC that takes a sequence of training checkpoints $\mathcal{C}^{(0)}, \ldots, \mathcal{C}^{(T)}$, ordered from oldest to newest, as input and produces a bit-wise lossless archive. ALC keeps only the latest checkpoint $\mathcal{C}^{(T)}$ uncompressed as the anchor, while storing each earlier checkpoint $\mathcal{C}^{(t)}$ as a compressed representation $\hat{\mathcal{C}}^{(t)}$ with respect to its immediate successor $\mathcal{C}^{(t+1)}$. During decompression, reconstruction starts from the anchor checkpoint and proceeds backward along the delta chain to recover historical states.

For each checkpoint pair $(\mathcal{C}_p, \mathcal{C}_c)$, ALC applies the adaptive pipeline shown in Algorithm 2 and Figure 11. The framework consists of three components. First, it removes losslessly recoverable redundancy, such as BF16 weights derivable from FP32 master weights. Second, it performs per-tensor adaptive path selection: based on cost estimates computed from a deterministic subsample, ALC chooses the lowest-cost path among one direct path (I-Align) and two delta paths (PCDelta and RotDelta). Third, it encodes the selected representation with a shared tANS-based backend. In this way, ALC jointly exploits structural redundancy within checkpoints and temporal redundancy across checkpoints, while allowing RotDelta to handle tensors affected by frequent sign flips.

Algorithm 2: Adaptive Compression Selection Pipeline

Input: checkpoints $\mathcal{C}^{(0)}, \mathcal{C}^{(1)}, \ldots, \mathcal{C}^{(T)}$; tANS encoder $\mathcal{M}$
Output: $\mathcal{C}^{(T)}$ uncompressed; compressed $\hat{\mathcal{C}}^{(0)}, \hat{\mathcal{C}}^{(1)}, \ldots, \hat{\mathcal{C}}^{(T-1)}$

Store $\mathcal{C}^{(T)}$ uncompressed as the head of the reverse delta chain
for $t = 1$ **to** T **do** ▷ *upon receiving $\mathcal{C}^{(t)}$, compress $\mathcal{C}^{(t-1)}$*
  ▷ *Phase 1: Redundancy Removal*
  Skip BF16 weight tensors from mixed-precision training

  for each remaining tensor $\mathbf{x}^{(t-1)} \in \mathcal{C}^{(t-1)}$ **do**
    Look up $\mathbf{x}^{(t)}$ of $\mathbf{x}^{(t-1)}$ in $\mathcal{C}^{(t)}$

    ▷ *Phase 2.a: Preprocessing*
    Draw deterministic subsample of $\min(n, 65536)$ elements
    $\mathbf{u}_{\text{align}} \leftarrow \text{RotMap}(\mathbf{x}^{(t-1)})$ ▷ *I-Align mapping*
    if $\mathbf{x}^{(t)}$ found **then**
      $\mathbf{v}_p^{\text{PCMap}} \leftarrow \text{PCMap}(\mathbf{x}^{(t)})$; $\mathbf{v}_c^{\text{PCMap}} \leftarrow \text{PCMap}(\mathbf{x}^{(t-1)})$
      $\mathbf{d}^{\text{PCMap}} \leftarrow |\mathbf{v}_c^{\text{PCMap}} - \mathbf{v}_p^{\text{PCMap}}|$ ▷ Δ-*PCMap* mapping
      $\mathbf{v}_p^{\text{RotMap}} \leftarrow \text{RotMap}(\mathbf{x}^{(t)})$; $\mathbf{v}_c^{\text{RotMap}} \leftarrow \mathbf{u}_{\text{align}}$
      $\mathbf{d}^{\text{RotMap}} \leftarrow |\mathbf{v}_c^{\text{RotMap}} - \mathbf{v}_p^{\text{RotMap}}|$ ▷ Δ-*RotMap* mapping

    ▷ *Phase 2.b: Compression Path Selection*
    $C_{\text{direct}}^{\text{I-Align}} \leftarrow \sum_{j=1}^{w} \hat{H}(B^{(j)})$
      ▷ *sum of per-column byte entropy, w = dtype byte width*
    if $\mathbf{x}^{(t)}$ not found **then** $m^* \leftarrow$ direct; **goto** Phase 3
    $C_{\text{delta}}^{\text{PCMap}} \leftarrow \hat{H}(S, K) + \hat{\mathbb{E}}[K]$ on $\mathbf{d}^{\text{PCMap}}$
    $C_{\text{delta}}^{\text{RotMap}} \leftarrow \hat{H}(S, K) + \hat{\mathbb{E}}[K]$ on $\mathbf{d}^{\text{RotMap}}$
    $m^* \leftarrow \text{argmin}_{m \in \{\text{I-Align, PCMap, RotMap}\}} C^m$
      ▷ *pick the lowest-entropy path*

    ▷ *Phase 2.c: Compression Encoder*
    if $m^* \in \{\text{PCMap, RotMap}\}$ **then** ▷ *delta path*
      for each $d \in \mathbf{d}^{m^*}$ **do**
        $S \leftarrow \text{sgn}(d)$; $\Delta \leftarrow |d|$
        if $\Delta > 0$ **then** $K \leftarrow \lfloor \log_2 \Delta \rfloor$ **else** $K \leftarrow 0$
        Encode joint symbol (S, K) via encoder $\mathcal{M}$
        Write the lowest K bits of Δ as raw bits
    else ▷ *direct path*
      Split $\mathbf{u}_{\text{align}}$ into w byte-columns
      Encode each byte-column via encoder $\mathcal{M}$
    Write compressed tensor $\hat{\mathbf{x}}^{(t-1)}$ with selected method m^*
return $\mathcal{C}^{(T)}$ uncompressed; compressed $\hat{\mathcal{C}}^{(0)}, \ldots, \hat{\mathcal{C}}^{(T-1)}$

B. Redundancy Removal

Our strategy first removes structural redundancy from training checkpoints without information loss, thereby reducing compression overhead and storage footprint. BF16 tensors that are bit-recoverable from shape-compatible FP32 tensors via (1) are omitted and regenerated during reconstruction. This removal leads to $\sim$15% savings in storage (see §III-B). In LoRA-style or partial fine-tuning, frozen tensors identical across checkpoints are stored only once in the segment's uncompressed checkpoint, with earlier copies replaced by compact references. This eliminates recoverable redundancy before tensor-level compression and further reduces the final archive size.

C. Compression Path Design

Our strategy provides two compression paths to exploit complementary forms of redundancy. The *direct path* targets intra-checkpoint spatial redundancy on a single tensor, while the *delta path* targets cross-checkpoint temporal redundancy on a delta against the next checkpoint. Both paths share a tANS encoder [42]; they differ in how the input stream is decomposed to exploit its data characteristics.

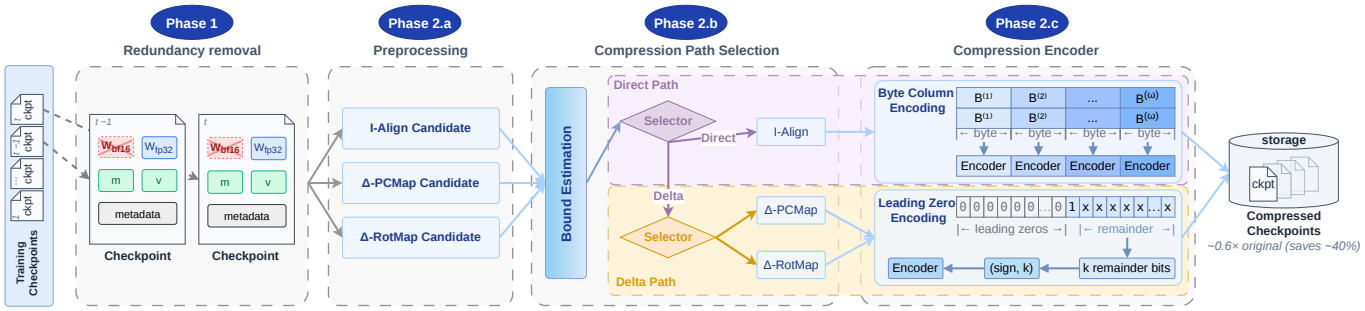


Fig. 11: ALC overview.

Direct Path. As analyzed in Section III-C, the direct path exploits the low-entropy structure of tensor data. Specifically, bytes at the same position across elements share similar statistics; for instance, exponent bits exhibit low entropy and a concentrated distribution. To expose this structure, we design an *I-Align* operation, which applies RotMap to move the sign bit to the least-significant position, aligning exponents with byte boundaries (Algorithm 2, Line 11). We then split each element into w byte columns (w=4 for FP32, w=2 for BF16) and compress each column independently with tANS (Algorithm 2, Lines 33–34).

Delta Path. As analyzed in Section III-D, adjacent checkpoints are temporally correlated, yielding deltas with smaller magnitudes and lower entropy. To leverage this insight, the delta path provides two algorithms, *PCDelta* and *RotDelta*. RotDelta applies RotMap to both checkpoints and takes their difference in the rotated integer domain, producing Δ_{RotMap} (Algorithm 2, Lines 15–16). PCDelta does the same with PCMap, producing Δ_{PCMap} (Algorithm 2, Lines 13–14). The signed deltas are then fed to the leading-zero encoder of Section IV-B, which decomposes each Δ into a sign bit S, the position K of its highest set bit, and the following K raw bits. As magnitude differences shrink, (S, K) becomes more concentrated, letting tANS approach its joint entropy (Algorithm 2, Lines 28–31).

D. Adaptive Path Selection

As characterized in Section III-C, master weights, m_t, and v_t exhibit distinct statistical characteristics, making it difficult to identify a single universally optimal algorithm. We therefore adopt adaptive path selection, choosing different algorithms for different tensor pairs to fully realize compression potential. To drive this selection, we build the cost model that estimates each algorithm's per-element bit-cost, rather than committing to a fixed algorithm.

Cost Model. As shown in Algorithm 2 (Lines 19–22), since the direct path encodes each of the w byte columns independently via tANS, its per-element cost is the sum of per-column entropies:

$$C_{\text{direct}}^{\text{I-Align}} = \sum_{j=1}^{w} H(B^{(j)}), \tag{12}$$

where $B^{(j)}$ denotes the byte column at position j.

Since the delta path decomposes each delta into a joint symbol (S, K) encoded by tANS and K raw remainder bits concatenated directly, its per-element cost is:

$$C_{\text{delta}} = H(S, K) + \mathbb{E}[K], \tag{13}$$

where $\mathbb{E}[K]$ is the expected number of raw remainder bits per element.

Selection Rule. As shown in Algorithm 2 (Line 23), the selector picks the algorithm with the lowest estimated cost on each mapped representation:

$$m^* = \arg \min_{m \in \{\text{I-Align, PCMap, RotMap}\}} C^m, \tag{14}$$

avoiding trial compression. The chosen strategy is recorded in the block header for the decompressor. To accelerate cost estimation, we subsample each tensor at $0.01\,‰$ rather than scanning all elements; for small tensors where this yields fewer than 65,536 samples, we evaluate all elements.

VI. Evaluation

In this section, we evaluate the empirical superiority, design soundness, production viability, and economic efficiency of ALC. We aim to answer the following research questions:

- **RQ1: Overall Performance.** How does ALC improve end-to-end compression ratio and throughput compared to baselines across diverse LLM configurations? (§VI-B)
- **RQ2: Breakdown Analysis.** How do individual components (e.g., adaptive selection, RotDelta) contribute to the overall gains and align with theoretical bounds? (§VI-C)
- **RQ3: Real-world Deployment.** How effective is ALC when applied to production-scale checkpoints? (§VI-D)
- **RQ4: Economic Analysis.** What is the storage-compute trade-off, and does the approach provide cost-efficient long-term archival? (§VI-E)

A. Experimental Setup

Setup. All experiments are conducted on a server equipped with an Intel Xeon Platinum 8469C processor (3.0 GHz) and 234 GB of RAM. Our scheme is implemented using Python 3.11 and PyTorch 2.10 (CUDA 12.8). The core routines for direct/delta encoding and byte operations are implemented in C++, while checkpoint loading, tensor traversal, and compression scheme selection are handled in Python.

Models and training. We evaluate six mainstream model architectures spanning diverse design paradigms: Qwen3.5-35B-A3B (hybrid attention Sparse Mixture-of-Experts), DeepSeek-7B (dense Transformers), Gemma4-E4B (dense Transformers with Per-Layer Embeddings), Mamba-2.8B (State Space Models), RetNet-2.7B (Retention Networks), and RWKV-7-2.9B (Attention-free RNNs). For each architecture, we perform full-parameter, mixed-precision training with the AdamW optimizer under DeepSpeed ZeRO Stage 2 on NVIDIA H20 GPUs. The complete training state, including BF16 model weights, FP32 master weights, and the first- and second-moment estimates (m_t, v_t), is periodically checkpointed at saving intervals $\Delta\text{step} \in \{100, 500, 1000\}$, with each training rank independently persisting its corresponding shard.

Baselines. We compare our scheme against six lossless compression baselines: (1) FM-Delta [21], which applies Δ-PCMap encoding followed by range coding uniformly across all parameters; (2) ZipLLM [22], which computes bitwise XOR deltas and applies Zstd compression; (3) ZipNN [20], which separates IEEE 754 byte fields and Huffman-codes the exponent stream; (4) Zstd [23], which directly compresses the raw data bytes without delta encoding; (5) range coding [31], which applies a general-purpose entropy coder to the raw data stream; and (6) FastDelta [35], a general-purpose delta compressor for backup systems that exploits rolling-hash reuse. All methods are verified by bitwise compress-and-decompress checks.

B. Overall Performance

We evaluate ALC's storage and runtime effectiveness against the baselines under representative LLM workloads. Specifically, we benchmark compression ratio, throughput, and robustness across diverse architectures, model scales, and saving intervals (Δstep). ALC consistently outperforms all baselines, demonstrating its viability for production-scale checkpointing.

Baseline comparison. Table I summarizes the compression ratio and throughput across the six models. Our scheme attains the lowest compression ratio under every configuration: it beats the strongest baseline FM-Delta by 5.6–11.4 percentage points (pp), and reduces the storage footprint by up to 34 pp against general-purpose and neural-network-specific compressors (ZipLLM, ZipNN, Zstd), reaching 50.2% on Gemma4-E4B at $\Delta\text{step} = 100$. On throughput, our scheme reaches 198.4 MB/s for compression and 232.5 MB/s for decompression. By contrast, FastDelta performs poorly because it cannot effectively capture the numerical similarity structure.

To investigate the exact source of our compression efficiency, Figures 12 provide a detailed breakdown across different checkpoint components with $\Delta\text{step} = 100$. Our structural redundancy removal eliminates the redundant BF16 model weights, directly saving 10%–15% of storage across all architectures. Beyond this structural saving, our adaptive compression scheme further reduces the compression ratio of FP32 optimizer states. Moreover, compared with FM-Delta, our scheme consistently improves the overall compression ratio of FP32 master weights and optimizer states by approximately 3%. Unless stated otherwise, the compression ratios reported in

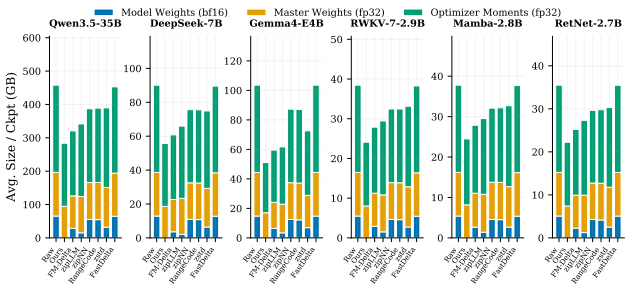


Fig. 12: Compression breakdown by checkpoint component ($\Delta\text{step} = 100$).

subsequent subsections are measured solely on the FP32 master weights and optimizer states, excluding the gains from structural redundancy removal. The breakdown with $\Delta\text{step} = 500$ is showed in Appendix B1 [43].

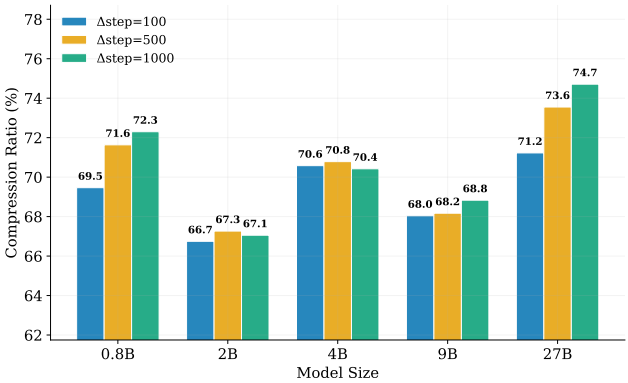


Fig. 13: Compression ratio across model sizes for different checkpoint intervals (Δstep) on the Qwen3.5 model family.

Scaling with model size. Beyond architectural family, model parameter count is another determinant of compression efficiency. Using the Qwen3.5 family as a representative case, Figure 13 reports the average compression ratio across five scales from 0.8B to 27B parameters under all saving intervals. The compression ratio stays within a 66.7%–74.7% band over a 30× scale span; at the largest scale (27B), our scheme reaches 71.2% at $\Delta\text{step} = 100$. Overall, our scheme exhibits stable scale robustness, with no per-scale tuning required.

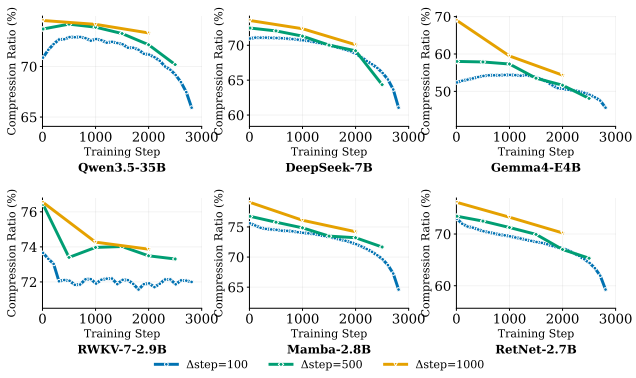


Fig. 14: Compression ratio vs. training progress under different checkpoint saving intervals ($\Delta\text{step} \in \{100, 500, 1000\}$). Lower values indicate better compression.

TABLE I: Total storage and compression ratio (%) across model architectures and delta steps. Total training steps = 3000. Orig. shows total uncompressed size of all checkpoints. Compression ratio = compressed total / original total ×100. Lower is better. All methods are lossless.

Δ Step	Model	Num.	Orig. (GB)	Ours	FM-Delta [21]	ZipLLM [22]	ZipNN [20]	Range coding [31]	Zstd [23]	FastDelta [35]
100	Qwen3.5-35B-A3B	30	13732	**62.0**	70.0	74.6	84.5	85.0	85.1	98.9
	DeepSeek-7B	30	2703	**61.8**	67.4	73.2	84.0	83.8	83.1	99.4
	Gemma4-E4B	30	3106	**50.2**	57.5	59.6	84.3	84.1	70.1	100.0
	RWKV-7-2.9B	30	1154	**62.6**	72.5	76.6	84.4	84.4	86.2	99.5
	Mamba-2.8B	30	1133	**64.8**	73.7	78.1	85.0	85.2	86.5	99.9
	RetNet-2.7B	30	1065	**62.6**	71.0	77.0	83.5	83.9	85.4	99.9
500	Qwen3.5-35B-A3B	6	2746	**66.8**	76.9	80.2	86.7	87.0	87.2	99.3
	DeepSeek-7B	6	541	**65.1**	72.8	77.3	86.2	86.1	85.5	99.4
	Gemma4-E4B	6	621	**55.1**	64.6	65.7	86.5	83.9	74.2	100.0
	RWKV-7-2.9B	6	231	**65.8**	76.5	79.9	86.5	86.5	88.1	99.5
	Mamba-2.8B	6	227	**67.7**	79.1	81.9	87.1	87.2	88.4	99.3
	RetNet-2.7B	6	213	**65.8**	76.4	80.7	85.8	83.9	87.4	99.6
Throughput (MB/s)										
	Compress	—	—	198.4	121.3	397.8	355.3	217.0	523.7	72.0
	Decompress	—	—	232.5	86.8	543.8	404.8	164.0	584.9	1780.0

Impact of Training Progress and Saving Intervals. We next examine the effect of temporal dynamics on compression ratio along two axes: training progress and checkpoint saving interval Δstep (Figure 14). First, the compression ratio decreases monotonically over training, by 1.5% to 14% from early to late stages across all architectures, as FP32 master weights and optimizer states gradually stabilize and successive checkpoints become more similar. Second, once training has stabilized, increasing Δstep from 100 to 1000 (a 10× increase) increases the compression ratio by only 1% to 5%, suggesting that our scheme exploits long-range correlations rather than depending solely on short-range similarity between adjacent checkpoints. Overall, our scheme remains temporally robust without requiring per-stage or per-interval tuning.

C. Breakdown Analysis

This section evaluates the design and effectiveness of each module in ALC. To keep the analysis clear, we focus only on the states that need compression: the FP32 master weights, the first-moment estimate m_t, and the second-moment estimate v_t. Using a representative large language model (such as Qwen3.5-9B) as our benchmark, we test ALC in four steps. First, we show why adaptive compression scheme selection is necessary across different checkpoints. Second, we compare RotDelta and FM-Delta to show the advantages of RotDelta. Third, we verify the actual compression performance matches our theoretical cost bounds. Finally, we evaluate the effectiveness of the subsampling strategy.

Necessity of Adaptive Compression Scheme Selection. The effectiveness of our adaptive compression scheme stems from parameter heterogeneity. As shown in Figure 15, the optimal scheme varies across tensors and training stages: smaller delta steps favor delta compression, while larger ones favor direct compression. Tensor behavior also differs: m_t benefits most from Δ-RotMap because it avoids sign-flip penalties, v_t is less suitable for Δ-PCMap because it is non-negative, and FP32 master weights favor Δ-PCMap due to their small increments and rare sign flips. Selecting the lowest-cost scheme for each tensor at each checkpoint therefore maximizes the overall compression ratio.

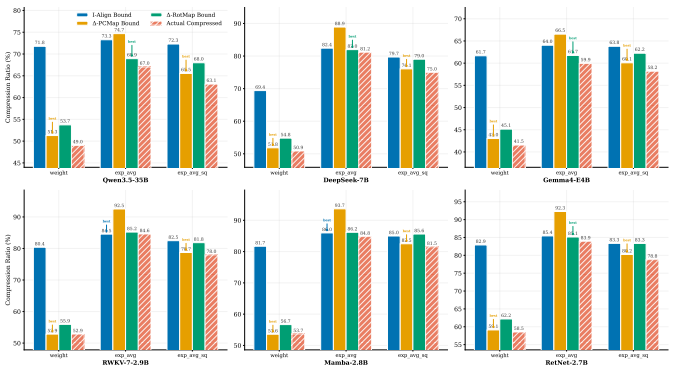
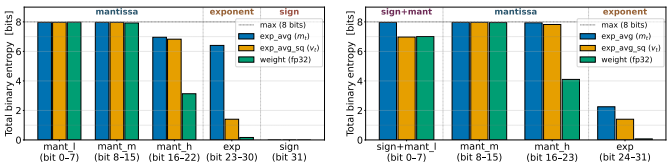


Fig. 15: Per-tensor theoretical bounds and actual compression bit-rates of Δ-RotMap, Δ-PCMap, and I-Align across all evaluated models (Δstep = 100).

Benefits of RotDelta under Sign Flips. To investigate the advantage of our RotDelta, we use a checkpoint pair (delta step = 100) from the Qwen3.5-9B model as an example. We analyze the delta value distributions after applying Δ-RotMap and Δ-PCMap transformations. The entropy distribution of the delta magnitude is shown in Figure 16, and the BSR distribution of the delta values is shown in Figure 17. First, compared to the raw data, both Δ-RotMap and Δ-PCMap reduce the entropy to varying degrees, providing more room for compression. Second, Δ-RotMap significantly reduces the exponent entropy of m_t. This indicates a higher probability of leading zeros, which is confirmed by Figure 17. Specifically, the BSR distribution of m_t in Figure 17 shows a dramatic decrease in data with high BSR values between 26 and 30. Since our compression scheme uses an encoder to compress leading zeros, the Δ-RotMap transformation naturally leads

to a smaller compressed size. This is further supported by Figure 15 where RotDelta achieves about a 7% improvement in compression performance over FM-Delta for deltas with frequent sign flips.



(a) FM-Delta (Δ-PCMap) (b) RotDelta (Δ-RotMap)

Fig. 16: Byte-level entropy distribution of the first-moment estimation m_t, v_t, and weights. (a) Results of the FM-Delta algorithm after Δ-PCMap, categorized by sign bit, exponent, and mantissa. (b) Byte-wise entropy values of the RotDelta algorithm after Δ-RotMap.

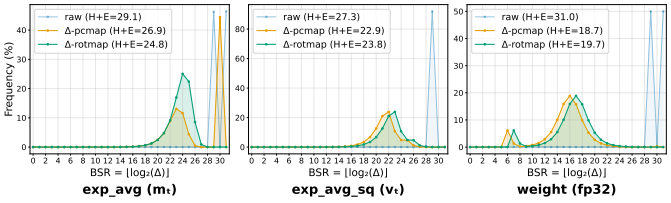


Fig. 17: BSR distributions of the raw data, Δ-PCMap, and Δ-RotMap. The horizontal axis represents the displacement rank measured by Bit Scan Reverse (BSR), which denotes the position of the most significant bit (MSB) of Δ; a smaller BSR value indicates higher compressibility of the delta. The legend presents the entropy bound $H + E$ (bits/element) for each element.

Tightness of Cost Bounds. To validate the accuracy of our cost models defined in Equation 12 and Equation 13, we compare the theoretically predicted bitrates of the three schemes with their empirical compression results. As shown in Figure 18, the gap between the predicted and actual bitrates consistently remains minimal across all tensor categories, demonstrating the high fidelity of our cost models in driving strategy selection. Specifically, for I-Align, this gap is approximately 3–5%, as its byte-level encoding operates on a 256-symbol alphabet and thus requires more samples to achieve convergence. In contrast, the gap for PCMap and RotMap is restricted to around 1%, because their smaller 65-symbol alphabet enables the encoder to rapidly converge to accurate probability estimations. These results robustly demonstrate that our cost models and theoretical bounds serve as highly reliable indicators for predicting and optimizing practical compression behaviors.

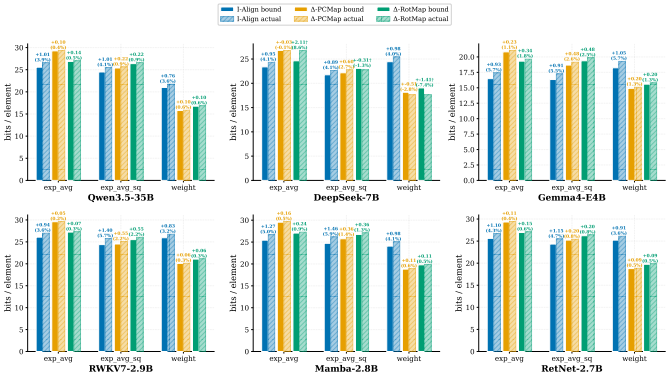


Fig. 18: Comparison of theoretical compression bounds and actual compression ratios.

Accuracy and Efficiency of Subsampling. To drive the adaptive selector using deterministic sub-samples rather than full tensors, we validate the relationship between sampling rate and estimation accuracy to guarantee high fidelity. As shown in Figure 19, the estimated theoretical compression ratio converges rapidly with the sampling rate: even at a sampling rate of merely 0.01‰ on a tensor with 2.35 billion elements, the sample-based estimate is virtually indistinguishable from the full-tensor baseline. Crucially, the entire estimation process completes in less than a millisecond, boosting computational efficiency by 1000×. For small tensors below the 64K-element threshold, we directly fall back to full-tensor evaluation, which inherently incurs negligible overhead.

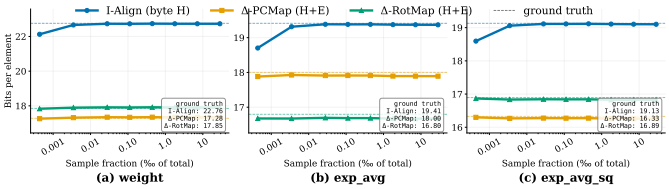


Fig. 19: Convergence of the sample-based theoretical-compression-rate estimate as a function of sampling ratio on Qwen3.5-9B ($\Delta\text{step} = 100$). The estimate converges to the full-tensor reference at a sampling ratio of 0.01‰, with sub-millisecond evaluation time.

D. Real-World Deployment

To validate that our compression gain generalizes to real-world production scenarios, we evaluate the proposed adaptive strategy on the production storage infrastructure at Alibaba Group. The evaluation covers Qwen models ranging from 8B to 235B parameters. As demonstrated in Table II, our method consistently outperforms all compared lossless baselines across these production checkpoints, achieving compression ratios of 56.4%–62.7% and beating the strongest baseline FM-Delta by 7–10 percentage points. In terms of absolute storage savings, the total uncompressed volume across five production models is approximately 49 TB; our strategy reduces this to roughly 29 TB, saving over 20 TB of persistent storage. These results confirm that ALC remain effective on production-scale check-

TABLE II: Total storage and compression ratio (%) for Pangu checkpoints. Orig. shows total uncompressed size of all checkpoints. Compression ratio = compressed total / original total $\times 100$. Lower is better. All methods are lossless.

Δ Step	Model	Num.	Orig. (GB)	Ours	FM-Delta [21]	ZipLLM [22]	ZipNN [20]	Range coding [31]	Zstd [23]	FastDelta [35]
2	Qwen2.5-72B	12	11,376	**60.9**	69.1	76.8	81.9	80.8	85.2	98.6
5	Qwen3-MoE-235B	5	15,959	**58.1**	67.1	73.0	82.0	81.2	79.2	98.0
10	Qwen3-MoE-30B	41	16,186	**56.4**	66.5	70.8	80.7	80.2	78.5	99.3
10	Qwen3.5-27B	18	6,436	**62.7**	71.4	73.4	81.4	81.8	81.4	98.5
22	Qwen3-VL-8B	2	216	**59.1**	67.5	71.9	80.2	82.0	84.5	98.7

points that are substantially larger and more heterogeneous than the controlled benchmarks. The strategy requires no per-model tuning and integrates transparently into existing storage.

E. Economic Benefit of ALC

The practical value of lossless checkpoint compression ultimately reduces to whether the CPU cycles spent on compression are cheaper than the persistent storage they save. Since checkpoints are written frequently but read only during rare training recovery events (especially after training pipeline finished), we focus on compression costs and neglect the negligible decompression overhead. Let V denote the raw checkpoint volume, r the achieved compression ratio representing the compressed size divided by the raw size, B_c the per core compression throughput, p_c the unit price of CPU time, p_s the unit price of persistent storage per unit time, and τ the retention horizon. The single instance CPU cost incurred when compressing a single checkpoint is

$$C_{\text{cpu}} \;=\; \frac{V}{B_c} \cdot p_c, \tag{15}$$

while the cumulative storage cost saved over the retention horizon is

$$C_{\text{stor}} \;=\; (1-r) \cdot V \cdot p_s \cdot \tau. \tag{16}$$

Compression is economically beneficial whenever $C_{\text{stor}} > C_{\text{cpu}}$, which yields the break even retention horizon

$$\tau^\star \;=\; \frac{p_c}{(1-r) \cdot B_c \cdot p_s}. \tag{17}$$

Equivalently, the return on investment ratio of compression is

$$R \;=\; \frac{C_{\text{stor}}}{C_{\text{cpu}}} \;=\; \frac{(1-r) \cdot B_c \cdot p_s \cdot \tau}{p_c}. \tag{18}$$

Equation 17 formalizes a clean substitution rule where every percentage point reduction in r proportionally shortens $\tau^\star$, and every multiplicative gain in B_c has the identical effect. Since training checkpoints are routinely retained on cold tier storage for durations much longer than $\tau^\star$, the linear amplification in Equation 18 explains why even moderate gains in the saving rate or throughput translate into order of magnitude reductions in archival cost.

Against the background of rising storage prices, we further simulate the economic benefit of ALC checkpoint compression under the above model. Using a fixed compression throughput of 200 MB/s, a representative compression ratio of $r = 0.6$ (i.e., 40% storage savings), and a CPU price of 1 yuan per core-day, compressing a 1 PB checkpoint on a single CPU core costs about 57.9 yuan. Table III shows that even under a broad range of storage prices, the break-even period remains short, indicating that checkpoint compression quickly recovers its compute cost once storage capacity is retained for archival.

TABLE III: Break-even period for compressing a 1 PB checkpoint under different storage prices. We assume a compression ratio of $r = 0.6$, a throughput of 200 MB/s, and a CPU price of 1 yuan per core-day.

Storage Price (yuan/GB/month)	Monthly Storage Saving (yuan/month)	Break-even Period $\tau^\star$
0.05	20,000	0.0868 days (2.08 h)
0.10	40,000	0.0434 days (1.04 h)
0.50	200,000	0.0087 days (12.5 min)
1.00	400,000	0.0043 days (6.3 min)

F. Discussions on the Generality of the Proposed Strategy

Although ALC focus on training with Adam-family optimizers, the underlying mechanisms (sign-flip behavior of the first-moment estimate m_t, low-entropy exponent fields, and inter-checkpoint correlation of FP32 master weights) are intrinsic to gradient-based optimization rather than to a specific objective. We therefore expect the strategy to generalize to other momentum-based optimizers (e.g., Muon [44]) and emerging reduced-precision formats such as FP8.

VII. Conclusion

We introduced a lossless compression framework ALC for LLM training checkpoints that exploits both spatial structures and temporal correlations. To overcome the sign-flip bottleneck of first-order momentum, we proposed RotDelta, a sign-agnostic delta scheme with a constant 1-bit overhead bound. This scheme is integrated into an entropy-guided, per-tensor adaptive framework ALC that selects optimal compression paths and eliminates structural redundancies, such as derivable BF16 weights. Across five LLM architectures, ALC consistently outperforms six lossless baselines, validating the efficiency of a structurally aware checkpoint storage design.

References

[1] B. Wan, M. Han, Y. Sheng, Y. Peng, H. Lin, M. Zhang, Z. Lai, M. Yu, J. Zhang, Z. Song, X. Liu, and C. Wu, "Bytecheckpoint: A unified checkpointing system for large foundation model development," in *22nd USENIX Symposium on Networked Systems Design and Implementation, NSDI 2025, Philadelphia, PA, USA, April 28-30, 2025*, T. A. Benson and R. N. Mysore, Eds. USENIX Association, 2025, pp. 559–578. [Online]. Available: https://www.usenix.org/conference/nsdi25/presentation/wan-borui

[2] Z. Wang, Z. Jia, S. Zheng, Z. Zhang, X. Fu, T. S. E. Ng, and Y. Wang, "GEMINI: fast failure recovery in distributed training with in-memory checkpoints," in *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, J. Flinn, M. I. Seltzer, P. Druschel, A. Kaufmann, and J. Mace, Eds. ACM, 2023, pp. 364–381. [Online]. Available: https://doi.org/10.1145/3600006.3613145

[3] J. Mohan, A. Phanishayee, and V. Chidambaram, "Checkfreq: Frequent, fine-grained DNN checkpointing," in *19th USENIX Conference on File and Storage Technologies, FAST 2021, February 23-25, 2021*, M. K. Aguilera and G. Yadgar, Eds. USENIX Association, 2021, pp. 203–216. [Online]. Available: https://www.usenix.org/conference/fast21/presentation/mohan

[4] R. Zhang, W. Xiao, H. Zhang, Y. Liu, H. Lin, and M. Yang, "An empirical study on program failures of deep learning jobs," in *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*, G. Rothermel and D. Bae, Eds. ACM, 2020, pp. 1159–1170. [Online]. Available: https://doi.org/10.1145/3377811.3380362

[5] H. Xiao, S. Yang, K. Zhang, Y. Jing, Z. He, and X. S. Wang, "Genie: A lightweight serverless infrastructure for in-memory key-value caching with fine-grained and prompt elasticity," *IEEE Transactions on Knowledge and Data Engineering*, 2025.

[6] B. Wan, G. Liu, Z. Song, J. Wang, Y. Zhang, G. Sheng, S. Wang, H. Wei, C. Wang, W. Lou, X. Yang, M. Zhang, K. Jiang, C. Ren, X. Zhi, M. Yu, Z. Nan, Z. Zheng, B. Zhong, Q. Wang, H. Yu, J. Chi, W. Zhang, Y. Li, Z. Du, S. Zhao, Y. Zhang, J. Tang, Z. Liu, C. Wu, Y. Peng, H. Lin, W. Xiao, X. Liu, and L. Xiang, "Robust LLM training infrastructure at bytedance," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP 2025, Lotte Hotel World, Seoul, Republic of Korea, October 13-16, 2025*, Y. Won, Y. Kwon, D. Yuan, and R. Isaacs, Eds. ACM, 2025, pp. 186–203. [Online]. Available: https://doi.org/10.1145/3731569.3764838

[7] J. Hu, F. Jin, X. Zhou, K. Zhang, Z. Shen, and Y. Luo, "Harp: Improving {VPC} network availability via efficient failure detection and rerouting in tencent cloud," in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, 2026, pp. 1669–1684.

[8] M. Gossman, A. Maurya, B. Nicolae, and J. Calhoun, "Understanding llm checkpoint/restore i/o strategies and patterns," in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region Workshops*, 2026, pp. 263–273.

[9] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2015. [Online]. Available: http://arxiv.org/abs/1412.6980

[10] I. Loshchilov, F. Hutter *et al.*, "Fixing weight decay regularization in adam," *arXiv preprint arXiv:1711.05101*, vol. 5, no. 5, p. 5, 2017.

[11] G. Sivathanu, C. P. Wright, and E. Zadok, "Ensuring data integrity in storage: techniques and applications," in *Proceedings of the 2005 ACM Workshop On Storage Security And Survivability, StorageSS 2005, Fairfax, VA, USA, November 11, 2005*, V. Atluri, P. Samarati, W. Yurcik, L. Brumbaugh, and Y. Zhou, Eds. ACM, 2005, pp. 26–36. [Online]. Available: https://doi.org/10.1145/1103780.1103784

[12] S. Maneas, K. Mahdaviani, T. Emami, and B. Schroeder, "A study of SSD reliability in large scale enterprise storage deployments," in *18th USENIX Conference on File and Storage Technologies, FAST 2020, Santa Clara, CA, USA, February 24-27, 2020*, S. H. Noh and B. Welch, Eds. USENIX Association, 2020, pp. 137–149. [Online]. Available: https://www.usenix.org/conference/fast20/presentation/maneas

[13] E. Xu, M. Zheng, F. Qin, Y. Xu, and J. Wu, "Lessons and actions: What we learned from 10k ssd-related storage system failures," in *Proceedings of the 2019 USENIX Annual Technical Conference, USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019*, D. Malkhi and D. Tsafrir, Eds. USENIX Association, 2019, pp. 961–976. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/xu

[14] W. Li, X. Chen, H. Shu, Y. Tang, and Y. Wang, "Excp: Extreme LLM checkpoint compression via weight-momentum joint shrinking," in *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. A. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds. PMLR / OpenReview.net, 2024, pp. 27 575–27 588. [Online]. Available: https://proceedings.mlr.press/v235/li24m.html

[15] A. Agrawal, S. Reddy, S. Bhattamishra, V. P. S. Nookala, V. Vashishth, K. Rong, and A. Tumanov, "Inshrinkerator: Compressing deep learning training checkpoints via dynamic quantization," in *Proceedings of the 2024 ACM Symposium on Cloud Computing, SoCC 2024, Redmond, WA, USA, November 20-22, 2024*. ACM, 2024, pp. 1012–1031. [Online]. Available: https://doi.org/10.1145/3698038.3698553

[16] Z. Wang, W. Zhu, Z. Zhang, C. Yan, F. Guo, Y. Li, and Y. Xu, "Amber: Towards fast and space-efficient incremental checkpointing in large language model training," in *Proceedings of the 54th International Conference on Parallel Processing, ICPP 2025, San Diego, CA, USA, September 8-11, 2025*. ACM, 2025, pp. 678–688. [Online]. Available: https://doi.org/10.1145/3754598.3754606

[17] A. Dutta, S. Krishnan, N. Kwatra, and R. Ramjee, "Accuracy is not all you need," in *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/e0e956681b04ac126679e8c7dd706b2e-Abstract-Conference.html

[18] Z. Xu, A. Gupta, T. Li, O. Bentham, and V. Srikumar, "Beyond perplexity: Multi-dimensional safety evaluation of LLM compression," in *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, ser. Findings of ACL, Y. Al-Onaizan, M. Bansal, and Y. Chen, Eds. Association for Computational Linguistics, 2024, pp. 15 359–15 396. [Online]. Available: https://doi.org/10.18653/v1/2024.findings-emnlp.901

[19] K. Marchisio, S. Dash, H. Chen, D. Aumiller, A. Üstün, S. Hooker, and S. Ruder, "How does quantization affect multilingual llms?" in *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, ser. Findings of ACL, Y. Al-Onaizan, M. Bansal, and Y. Chen, Eds. Association for Computational Linguistics, 2024, pp. 15 928–15 947. [Online]. Available: https://doi.org/10.18653/v1/2024.findings-emnlp.935

[20] M. Hershcovitch, A. Wood, L. Choshen, G. Girmonsky, R. Leibovitz, O. Ozeri, I. Ennmouri, M. Malka, S. P. Chin, S. Sundararaman, and D. Harnik, "Zipnn: Lossless compression for AI models," in *18th IEEE International Conference on Cloud Computing, CLOUD 2025, Helsinki, Finland, July 7-12, 2025*, R. N. Chang, C. K. Chang, J. Yang, N. Atukorala, D. Chen, S. Helal, S. Tarkoma, Q. He, T. Kosar, C. A. Ardagna, F. Awaysheh, V. Hilt, and Y. Simmhan, Eds. IEEE, 2025, pp. 186–198. [Online]. Available: https://doi.org/10.1109/CLOUD67622.2025.00028

[21] W. Ning, J. Wang, Q. Qi, M. Zhu, H. Sun, D. Cheng, J. Liao, and C. Zhang, "Fm-delta: Lossless compression for storing massive fine-tuned foundation models," in *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/7b75a7339dfb256ee4b4bec028a6890b-Abstract-Conference.html

[22] Z. Wang, T. Lan, Z. Su, J. Yang, and Y. Cheng, "Zipllm: Efficient LLM storage via model-aware synergistic data deduplication and compression," in *23rd USENIX Symposium on Networked Systems Design and Implementation, NSDI 2026, Renton, WA, May 4-6, 2026*, S. Kandula and H. Weatherspoon, Eds. USENIX Association, 2026, pp. 2371–2387. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/wang-zirui

[23] Y. Collet and M. S. Kucherawy, "Zstandard compression and the application/zstd media type," vol. 8478, 2018, pp. 1–54. [Online]. Available: https://doi.org/10.17487/RFC8478

[24] P. Deutsch, "GZIP file format specification version 4.3," *RFC*, vol. 1952, pp. 1–12, 1996. [Online]. Available: https://doi.org/10.17487/RFC1952

[25] Y. Chen, Z. Liu, B. Ren, and X. Jin, "On efficient constructions of checkpoints," in *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, ser. Proceedings of Machine Learning Research. PMLR, 2020, pp. 1627–1636. [Online]. Available: http://proceedings.mlr.press/v119/chen20m.html

[26] A. Eisenman, K. K. Matam, S. Ingram, D. Mudigere, R. Krishnamoorthi, K. Nair, M. Smelyanskiy, and M. Annavaram, "Check-n-run: a checkpointing system for training deep learning recommendation models," in *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022, Renton, WA, USA, April 4-6, 2022*,

A. Phanishayee and V. Sekar, Eds. USENIX Association, 2022, pp. 929–943. [Online]. Available: https://www.usenix.org/conference/nsdi22/presentation/eisenman

[27] Z. Hu, X. Zou, W. Xia, S. Jin, D. Tao, Y. Liu, W. Zhang, and Z. Zhang, "Delta-dnn: Efficiently compressing deep neural networks via exploiting floats similarity," in *ICPP 2020: 49th International Conference on Parallel Processing, Edmonton, AB, Canada, August 17-20, 2020*, J. N. Amaral, L. K. John, and X. Shen, Eds. ACM, 2020, pp. 40:1–40:12. [Online]. Available: https://doi.org/10.1145/3404397.3404408

[28] L. Xiang, X. Lu, R. Zhang, and Z. Hu, "SSDC: A scalable sparse differential checkpoint for large-scale deep recommendation models," in *IEEE International Symposium on Circuits and Systems, ISCAS 2024, Singapore, May 19-22, 2024*. IEEE, 2024, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ISCAS58744.2024.10557880

[29] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[30] I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic coding for data compression," *Commun. ACM*, vol. 30, no. 6, pp. 520–540, 1987. [Online]. Available: https://doi.org/10.1145/214762.214771

[31] M. Schindler, "A fast renormalisation for arithmetic coding," in *Data Compression Conference, DCC 1998, Snowbird, Utah, USA, March 30 - April 1, 1998*. IEEE Computer Society, 1998, p. 572. [Online]. Available: https://doi.org/10.1109/DCC.1998.672314

[32] P. Lindstrom and M. Isenburg, "Fast and efficient compression of floating-point data," *IEEE Trans. Vis. Comput. Graph.*, vol. 12, no. 5, pp. 1245–1250, 2006. [Online]. Available: https://doi.org/10.1109/TVCG.2006.143

[33] M. Burtscher and P. Ratanaworabhan, "FPC: A high-speed compressor for double-precision floating-point data," *IEEE Trans. Computers*, vol. 58, no. 1, pp. 18–31, 2009. [Online]. Available: https://doi.org/10.1109/TC.2008.131

[34] K. Masui, M. Amiri, L. Connor, M. Deng, M. Fandino, C. Höfer, M. Halpern, D. Hanna, A. D. Hincks, G. Hinshaw, J. M. Parra, L. Newburgh, J. R. Shaw, and K. Vanderlinde, "A compression scheme for radio data in high performance computing," *Astron. Comput.*, vol. 12, pp. 181–190, 2015. [Online]. Available: https://doi.org/10.1016/j.ascom.2015.07.002

[35] H. Tan, W. Ou, X. Zou, C. Deng, Y. Pan, H. Huang, Z. Gu, and W. Xia, "Once rolling hashing is enough: Exploiting rolling hash reuse in delta compression," in *Proceedings of the 21st European Conference on Computer Systems, EuroSys 2026, McEwan Hall/The University of Edinburgh, Edinburgh, Scotland, UK, April 27-30, 2026*, A. Barbalace, L. Mai, R. Geambasu, and P. R. Pietzuch, Eds. ACM, 2026, pp. 919–933. [Online]. Available: https://doi.org/10.1145/3767295.3803596

[36] Y. Zhang, H. Jiang, D. Feng, N. Jiang, T. Qiu, and W. Huang, "{LoopDelta}: Embedding locality-aware opportunistic delta compression in inline deduplication for highly efficient data reduction," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 133–148.

[37] P. Micikevicius, S. Narang, J. Alben, G. F. Diamos, E. Elsen, D. García, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," vol. abs/1710.03740, 2017. [Online]. Available: http://arxiv.org/abs/1710.03740

[38] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: memory optimizations toward training trillion parameter models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, C. Cuicchi, I. Qualters, and W. T. Kramer, Eds. IEEE/ACM, 2020, p. 20. [Online]. Available: https://doi.org/10.1109/SC41405.2020.00024

[39] J. W. Young, "A first order approximation to the optimum checkpoint interval," *Commun. ACM*, vol. 17, no. 9, p. 530–531, Sep. 1974. [Online]. Available: https://doi.org/10.1145/361147.361115

[40] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, "Pytorch distributed: Experiences on accelerating data parallel training," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3005–3018, 2020. [Online]. Available: http://www.vldb.org/pvldb/vol13/p3005-li.pdf

[41] "Ieee standard for floating-point arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.

[42] J. Duda, "Asymmetric numeral systems," *CoRR*, vol. abs/0902.0271, 2009. [Online]. Available: http://arxiv.org/abs/0902.0271

[43] "Acl full version," https://github.com/jerryp1/ACL/blob/main/ACL_full.pdf.

[44] K. Jordan, Y. Jin, V. Boza, Y. Jiacheng, F. Cesista, L. Newhouse, and J. Bernstein, "Muon: An optimizer for hidden layers in neural networks," 2024. [Online]. Available: https://kellerjordan.github.io/posts/muon/

APPENDIX

A. Proof of Proposition 1

Proof. $\Delta_{\text{RotMap}} < \Delta_{\text{PCMap}}$ requires $2(L(x_p) - L(x_c)) + (s_p - s_c) < L(x_p) + L(x_c) + 1$. Since $|s_p - s_c| = 1$, a sufficient condition is

$$2(L(x_p) - L(x_c)) < L(x_p) + L(x_c), \tag{19}$$

equivalently $L(x_p) < 3\,L(x_c)$, together with $L(x_c) < L(x_p)$ implied by the WLOG assumption. Although $L(x)$ is defined by interpreting the IEEE 754 bit pattern of x as an integer, the sufficient condition admits a clean interpretation in the floating-point domain. Writing $L(x) = E(x) \cdot 2^{23} + M(x)$ with $M(x) < 2^{23}$, the condition $L(x_p) < 3\,L(x_c)$ reduces to leading order to $E(x_p) < 3E(x_c)$. Substituting $E(x) = e(x) + 127$ with unbiased exponent $e(x)$, the condition becomes $e(x_p) < 3e(x_c) + 254$. Therefore, the condition holds throughout the FP32 normal range, covering all values encountered in normal training. □

B. Extended Figures

1) Compression breakdown with Δstep $= 500$: Figure 20 provides a detailed breakdown across different checkpoint components with $\Delta\text{step} = 500$.

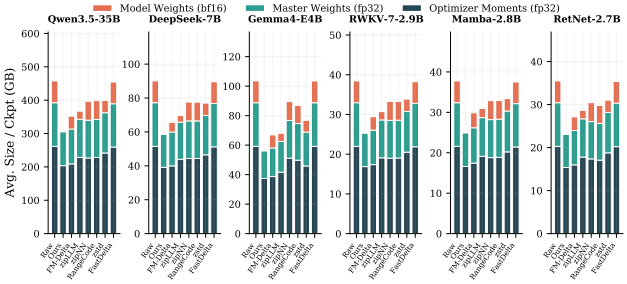


Fig. 20: Compression breakdown by checkpoint component ($\Delta\text{step} = 500$).